

Bellman-Ford: Distributed Shortest Path

Alessia Smeralda Brunello

Matricola: 544964

Hands-On 17

Indice

1	Introduzione al problema	2
2	Stato dell'arte	2
3	Metodologia usata	3
3.1	Rappresentazione del grafo	3
3.2	Algoritmo di bellman-ford	4
4	Risultati sperimentali	4
4.1	Output	4
5	Conclusione e sviluppi futuri	5

1 Introduzione al problema

L'obiettivo dell'esercitazione è implementare in linguaggio C una versione distribuita dell'algoritmo di Bellman-Ford per il calcolo dei cammini minimi su un grafo orientato e pesato. Il grafo considerato è composto da:

- 4 nodi
- 6 archi diretti

Ciascun arco è associato ad un peso. Il nodo sorgente è il nodo da cui parte il calcolo delle distanze minime verso tutti gli altri nodi.

L'aspetto centrale dell'esercizio non è soltanto l'applicazione dell'algoritmo di Bellman-Ford, ma la sua simulazione in un contesto distribuito:

Ogni nodo del grafo viene infatti rappresentato da un processo distinto, creato tramite `fork()`, e la comunicazione tra nodi avviene tramite FIFO, cioè named pipe messe a disposizione dal sistema operativo.

In questa implementazione ogni arco diretto del grafo corrisponde a una FIFO. Se esiste un arco dal nodo u al nodo v , allora il processo associato al nodo u può inviare messaggi al processo associato al nodo v attraverso la FIFO relativa a quell'arco. In questo modo la topologia del grafo viene riprodotta anche nella struttura di comunicazione tra processi.

2 Stato dell'arte

Il problema dei cammini minimi è centrale negli algoritmi sui grafi e trova applicazione in ambiti come reti di calcolatori, routing e sistemi distribuiti.

In questi contesti, il peso di un arco può rappresentare un costo, un ritardo o una distanza tra due nodi.

L'algoritmo di Bellman-Ford risolve il problema dei cammini minimi da una singola sorgente tramite rilassamenti successivi degli archi. Rispetto ad altri algoritmi, come Dijkstra, è più generale perché non richiede necessariamente pesi positivi.

Nel modello distribuito non esiste un unico processo che conosce l'intero grafo: ogni nodo mantiene solo il proprio stato locale, cioè la distanza attualmente nota dalla sorgente e il padre nel cammino minimo. Le distanze vengono aggiornate progressivamente grazie ai messaggi scambiati tra nodi vicini, secondo un principio simile ai protocolli di routing distance-vector.

3 Metodologia usata

Il progetto è stato organizzato in modo modulare, separando:

- la descrizione del grafo,
- la logica dell'algoritmo distribuito
- le funzioni comuni di supporto

In questo modo il codice risulta più leggibile e più semplice da modificare.

3.1 Rappresentazione del grafo

Nel file `graph.c` il grafo è rappresentato tramite un array di strutture `Edge`. Ogni arco contiene il nodo di partenza, il nodo di arrivo e il peso associato.

La numerazione adottata è la seguente:

- il nodo 0 rappresenta la sorgente,
- il nodo 1 il nodo in alto a destra,
- il nodo 2 il nodo in basso a sinistra
- il nodo 3 il nodo in basso a destra.

Gli archi implementati sono:

$$0 \rightarrow 1 (6), \quad 0 \rightarrow 2 (1), \quad 2 \rightarrow 3 (2), \quad 3 \rightarrow 0 (3), \quad 1 \rightarrow 3 (1), \quad 3 \rightarrow 1 (2)$$

La parte principale dell'algoritmo si trova in `bellman_ford_fifo.c`. Ogni nodo viene eseguito da un processo figlio mediante la funzione `node_process()`. All'inizio, il nodo sorgente inizializza la propria distanza a 0, mentre tutti gli altri nodi impostano la distanza a `INF`, valore che rappresenta l'assenza di un cammino noto.

L'algoritmo procede per round. In ogni round ciascun processo esegue tre fasi:

1. invia la propria distanza corrente sui propri archi uscenti;
2. riceve le distanze dai nodi collegati tramite archi entranti;
3. aggiorna il proprio stato locale se uno dei messaggi ricevuti permette di ottenere un cammino più breve.

3.2 Algoritmo di bellman-ford

La funzione `msg()` costruisce il messaggio da inviare ai vicini, contenente l'identificativo del nodo mittente e la sua distanza corrente.

La funzione `stf()` aggiorna invece lo stato del nodo ricevente.

La regola di aggiornamento utilizzata è quella classica di Bellman-Ford:

$$d(v) = \min(d(v), d(u) + w(u, v))$$

dove:

- $d(v)$ è la distanza corrente del nodo v ,
- $d(u)$ è la distanza comunicata dal nodo vicino u ,
- $w(u, v)$ è il peso dell'arco da u a v

Se il valore $d(u) + w(u, v)$ è minore della distanza attualmente nota, il nodo aggiorna la propria distanza e imposta u come padre nel cammino minimo.

4 Risultati sperimentali

Il programma è stato compilato tramite `make` e utilizzando il comando `make run` che contiene il comando `./bellman_ford` è stato eseguito.

4.1 Output

```
alexia23@aleb23:~/LabReSid24-25/hands-on/ho17/codice$ make run
./bellman_ford

Risultati Bellman-Ford distribuito
-----
Nodo 0: distanza = 0, padre = 0
Nodo 1: distanza = 5, padre = 3
Nodo 2: distanza = 1, padre = 0
Nodo 3: distanza = 3, padre = 2
```

I risultati sono coerenti con i cammini minimi attesi a partire dal nodo sorgente 0.

Il nodo 2 viene raggiunto direttamente dalla sorgente tramite l'arco $0 \rightarrow 2$ di peso 1.

Il nodo 3 viene raggiunto passando per il nodo 2, con costo complessivo $1 + 2 = 3$.

Infine, il nodo 1 viene raggiunto con costo minimo passando per il cammino $0 \rightarrow 2 \rightarrow 3 \rightarrow 1$, il cui costo totale è:

$$1 + 2 + 2 = 5$$

Questo cammino è preferibile rispetto all'arco diretto $0 \rightarrow 1$, che ha costo 6.

Dal punto di vista sperimentale, il programma conferma il corretto funzionamento della simulazione distribuita: ogni nodo aggiorna la propria distanza solo sulla base dei messaggi ricevuti dai vicini, senza accedere a una struttura globale delle distanze. La FIFO finale viene usata esclusivamente per raccogliere i risultati, mentre il calcolo vero e proprio avviene tramite le FIFO associate agli archi del grafo.

5 Conclusione e sviluppi futuri

L'esercitazione ha permesso di implementare una versione distribuita dell'algoritmo di Bellman-Ford usando processi e FIFO in linguaggio C.

Ogni nodo è stato simulato da un processo distinto e ogni arco del grafo da una FIFO, rendendo esplicito il modello di comunicazione tra nodi vicini.

I risultati ottenuti confermano il corretto funzionamento dell'algoritmo: attraverso scambi locali di messaggi, i nodi riescono a calcolare le distanze minime dalla sorgente.

Come possibile sviluppo futuro, il programma potrebbe essere esteso per leggere il grafo da file, gestire grafi di dimensione variabile e introdurre una sincronizzazione più strutturata tra i round.